JobKit - Playwright interview questions

98 questions for Playwright. Each answer is five pointers with working code. Single-file download.

1. What is Playwright?

1. Definition you should say first: A Microsoft library to automate Chromium, Firefox, and WebKit. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Playwright is Microsoft's open-source library for reliable end-to-end tests. One API covers Chromium, Firefox, and WebKit, with browsers installed via `npx playwright install`.

3. In interviews, contrast it with Selenium: auto-wait, traces, API mocking, and test fixtures are built in. Node/TS is the default at most web teams; Python uses the same browser engine.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npm init playwright@latest
npx playwright install
npx playwright test --project=chromium
npx playwright show-report
```

5. If you only say 'it automates browsers' you fail the next question. Name locators, contexts, traces, and storageState.

2. Playwright vs Selenium?

1. Definition you should say first: Auto-wait, browsers included, and a single API across engines. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Selenium talks WebDriver and often needs explicit waits. Playwright talks browser protocols, auto-waits for actionability, and ships browsers.

3. Migration talk: map drivers to browserType, By.css to getByRole, and TestNG suites to playwright.config projects. Keep business assertions; throw away Thread.sleep.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('textbox', { name: 'Email' }).fill('user@test.com');
await expect(page.getByText('Signed in')).toBeVisible();
// Selenium equivalent needed WebDriverWait + ExpectedConditions
```

5. Do not trash Selenium. Say where it still wins: Appium mobile, old IE, or a huge existing grid.

3. What is a browser context?

1. Definition you should say first: An isolated incognito-like session inside a browser. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A context is an isolated incognito-like profile: cookies, storage, permissions. One browser can host many contexts.

3. Playwright Test gives you a fresh context per test by default. Use storageState to inject a saved login into that context.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const context = await browser.newContext({
  storageState: 'playwright/.auth/user.json',
  locale: 'en-IN',
  viewport: { width: 1280, height: 720 },
});
const page = await context.newPage();
```

5. Sharing one context across tests leaks cookies and makes parallel runs flaky.

4. What is a page?

1. Definition you should say first: A tab inside a context. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A page is a tab. Almost every user action (goto, click, fill) hangs off page or a locator created from page.

3. Wait for popup pages when a click opens a tab. Never assume page.locator after a navigation without an assertion.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Open report' }).click(),
]);
await popup.waitForLoadState();
await expect(popup).toHaveURL(/report/);
```

5. Holding a page after context close throws. Isolate pages per test.

5. What is a locator?

JobKit - Playwright interview questions

1. Definition you should say first: A lazy handle to find elements, not a one-time query. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.
3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```

5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

6. Why are locators better than element handles?

1. Definition you should say first: They re-query and wait; handles go stale. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.
3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```

5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

7. What is getByRole?

1. Definition you should say first: Locate by accessible role and name. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. getByRole uses the accessibility tree: role plus accessible name. This matches how keyboard and screen-reader users see the page.
3. Use name as a string or regex. Add exact: true when two buttons share a prefix. Prefer this over CSS ids.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Submit application' }).click();
await page.getByRole('textbox', { name: 'Email' }).fill('a@b.com');
await page.getByRole('combobox', { name: 'Country' }).selectOption('IN');
```

5. Icon-only buttons without an accessible name cannot be found. Fix the app (aria-label) or fall back to test id.

8. What is getByTestId?

1. Definition you should say first: Locate by a test id attribute you control. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. getByTestId targets a stable hook you own, usually data-testid. Use it when roles collide or the UI is canvas-like.
3. Agree the attribute in playwright.config testIdAttribute. Do not sprinkle test ids on every div; prefer roles first.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
// playwright.config.ts
export default { use: { testIdAttribute: 'data-testid' } };
await page.getByTestId('order-total').click();
await expect(page.getByTestId('order-total')).toHaveText('INR 1,200');
```

5. If frontend renames test ids without telling QA, CI goes red. Treat them as a contract.

9. What is getByText?

1. Definition you should say first: Locate by visible text. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. getByText finds visible text nodes. Good for messages; bad for buttons (use getByRole) and for text that is translated.
3. Pass exact or regex. Combine with a parent locator so you do not hit a footer duplicate.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByText('Application submitted', { exact: true })).toBeVisible();
await page.locator('main').getByText(/saved at/i).click();
```

5. Marketing copy changes weekly. Prefer role+name or test id for actions.

10. What is a strict mode violation?

1. Definition you should say first: Locator resolved to more than one element. Then spend the rest of the answer on

architecture, a working example, and a production failure.
2. Strict mode fails when a locator resolves to more than one element. That is a feature: ambiguous locators hide bugs.
3. Tighten with getByRole name, filter({ hasText }), or nth(0) only when the list is intentional.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const row = page.getByRole('row').filter({ hasText: 'ORD-1042' });
await row.getByRole('button', { name: 'Edit' }).click();
await page.getByRole('listitem').nth(2).click();
```

5. nth(0) on a duplicated Submit button is a smell. Fix the locator or the DOM.

11. How do you pick one of many?

1. Definition you should say first: nth, filter, or a tighter locator. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Strict mode fails when a locator resolves to more than one element. That is a feature: ambiguous locators hide bugs.
3. Tighten with getByRole name, filter({ hasText }), or nth(0) only when the list is intentional.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const row = page.getByRole('row').filter({ hasText: 'ORD-1042' });
await row.getByRole('button', { name: 'Edit' }).click();
await page.getByRole('listitem').nth(2).click();
```

5. nth(0) on a duplicated Submit button is a smell. Fix the locator or the DOM.

12. What is auto-waiting?

1. Definition you should say first: Playwright waits for actionability before click or fill. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Before click/fill, Playwright waits until the element is visible, enabled, stable, and receiving events.
3. You rarely need waitForSelector. If an action times out, inspect intercepting overlays and disabled states in the trace.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Pay' }).click({ timeout: 15000 });
// waits for actionability, then clicks
```

5. force: true skips actionability and causes 'it clicked in CI but UI did nothing' bugs.

13. What is actionability?

1. Definition you should say first: Visible, enabled, stable, and receiving events. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Before click/fill, Playwright waits until the element is visible, enabled, stable, and receiving events.
3. You rarely need waitForSelector. If an action times out, inspect intercepting overlays and disabled states in the trace.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Pay' }).click({ timeout: 15000 });
// waits for actionability, then clicks
```

5. force: true skips actionability and causes 'it clicked in CI but UI did nothing' bugs.

14. What is a timeout in Playwright?

1. Definition you should say first: Default 30s for many actions; set per test or per action. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Default action and expect timeout is 30s (configurable). Test timeout is separate (often 30s-60s per test).
3. Set a realistic test timeout in config. Override per slow navigation. Never set 0 globally.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export default defineConfig({
  timeout: 60_000,
  expect: { timeout: 10_000 },
  use: { actionTimeout: 15_000, navigationTimeout: 30_000 },
});
```

5. A 5-minute timeout hides deadlocks. Fail fast, then add a targeted wait for a known spinner.

15. What is expect(locator).toBeVisible()?

1. Definition you should say first: An assertion that waits until the element is visible. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web-first assertions retry until they pass or time out. expect(locator).toBeVisible() is not a one-shot check.
3. Assert outcomes, not sleeps. Chain toHaveURL, toHaveText, toHaveCount after the action.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByRole('dialog')).toBeVisible();
await expect(page.getByRole('dialog')).toContainText('Confirm delete');
```

```
await expect(page.getByRole('dialog')).toBeHidden();
```

5. Non-retrying expect(await locator.textContent()).toBe() races the UI. Use locator assertions.

16. Web-first assertions?

1. Definition you should say first: Assertions that retry until they pass or time out. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web-first assertions retry until they pass or time out. expect(locator).toBeVisible() is not a one-shot check.

3. Assert outcomes, not sleeps. Chain toHaveURL, toHaveText, toHaveCount after the action.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByRole('dialog')).toBeVisible();
await expect(page.getByRole('dialog')).toContainText('Confirm delete');
await expect(page.getByRole('dialog')).toBeHidden();
```

5. Non-retrying expect(await locator.textContent()).toBe() races the UI. Use locator assertions.

17. What is a fixture?

1. Definition you should say first: Shared setup in Playwright Test, such as page. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Fixtures inject setup (page, logged-in page, API client). test.extend adds your own.

3. Put login, seed, and teardown in fixtures so tests stay one screen of business steps.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export const test = base.extend< { adminPage: Page } >({
  adminPage: async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'playwright/.auth/admin.json' });
    const page = await ctx.newPage();
    await use(page);
    await ctx.close();
  },
});
```

5. Heavy fixture work in every test without worker reuse will explode CI time.

18. What is test.extend?

1. Definition you should say first: Custom fixtures for login or data. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Fixtures inject setup (page, logged-in page, API client). test.extend adds your own.

3. Put login, seed, and teardown in fixtures so tests stay one screen of business steps.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export const test = base.extend< { adminPage: Page } >({
  adminPage: async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'playwright/.auth/admin.json' });
    const page = await ctx.newPage();
    await use(page);
    await ctx.close();
  },
});
```

5. Heavy fixture work in every test without worker reuse will explode CI time.

19. What is storageState?

1. Definition you should say first: Saved cookies/localStorage to skip login. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.

3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

20. How do you reuse login?

1. Definition you should say first: Save storageState in a setup project. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.
3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

21. What is a project in playwright.config?

1. Definition you should say first: A browser or setup target. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.
3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```

5. Parallel tests that write the same user row or the same download folder will collide.

22. What is a worker?

1. Definition you should say first: A parallel process that runs tests. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.
3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```

5. Parallel tests that write the same user row or the same download folder will collide.

23. What is fullyParallel?

1. Definition you should say first: Run tests in a file in parallel. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.
3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```

5. Parallel tests that write the same user row or the same download folder will collide.

24. How do you stop test pollution?

1. Definition you should say first: Isolated contexts and no shared mutable files. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Pollution is leftover state: cookies, users, files, clocks. Isolation is a context per test plus unique data.
3. Create data via API with a random suffix. Clean up in fixture teardown if the env is shared.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const email = `qa.${Date.now()}@mail.test`;
```

```
await request.post('/api/users', { data: { email, role: 'applicant' } });
```

5. A passed test that leaves an open dialog can fail the next test in headed debug only - still pollution.

25. What is test.step?

1. Definition you should say first: A named chunk in the report. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. test.step names a chunk in the HTML report and trace. Use it for login, fill, assert blocks.

3. Keep steps business-shaped: 'Submit ATS form', not 'click div 3'.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await test.step('submit form', async () => {
  await page.getByLabel('Notice period').fill('30');
  await page.getByRole('button', { name: 'Continue' }).click();
});
```

5. Nested steps 8 levels deep make traces unreadable.

26. What is a trace?

1. Definition you should say first: A recording you can open in Trace Viewer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.

3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```

5. Always-on traces in a 2000-test nightly will fill the agent disk.

27. When do you keep traces?

1. Definition you should say first: On first retry or on failure in CI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.

3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```

5. Always-on traces in a 2000-test nightly will fill the agent disk.

28. What is Trace Viewer?

1. Definition you should say first: UI to inspect DOM, network, and actions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.

3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```

5. Always-on traces in a 2000-test nightly will fill the agent disk.

29. What is codegen?

1. Definition you should say first: Record actions into a test script. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Codegen records clicks into a script. It is a start, not a suite.

3. Re-record, then replace CSS with roles, add assertions, extract POM/fixtures, delete waits.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright codegen https://careers.example.com
# then rewrite:
await page.getByRole('textbox', { name: 'Phone' }).fill('9999999999');
```

5. Shipping raw codegen is a common junior red flag: brittle selectors and no assertions.

30. Why not ship raw codegen?

1. Definition you should say first: It is a start; you still clean locators and asserts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Codegen records clicks into a script. It is a start, not a suite.

3. Re-record, then replace CSS with roles, add assertions, extract POM/fixtures, delete waits.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright codegen https://careers.example.com
# then rewrite:
await page.getByRole('textbox', { name: 'Phone' }).fill('9999999999');
```

5. Shipping raw codegen is a common junior red flag: brittle selectors and no assertions.

31. What is page.route?

1. Definition you should say first: Intercept and mock network requests. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. page.route intercepts requests. fulfill returns a stub body so UI tests do not hit real backends.
3. Mock only the unstable dependency. Let the app static assets load normally.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route('**/api/jobs*', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ jobs: [{ id: 'J1', title: 'Tosca Engineer' }] }),
  });
});
```

5. A mock that never expires will hide contract breaks. Pair with one real API smoke test.

32. How do you mock an API?

1. Definition you should say first: route fulfill with JSON. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. page.route intercepts requests. fulfill returns a stub body so UI tests do not hit real backends.
3. Mock only the unstable dependency. Let the app static assets load normally.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route('**/api/jobs*', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ jobs: [{ id: 'J1', title: 'Tosca Engineer' }] }),
  });
});
```

5. A mock that never expires will hide contract breaks. Pair with one real API smoke test.

33. How do you wait for a response?

1. Definition you should say first: page.waitForResponse with a URL predicate. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. waitForResponse ties the click to the network you care about, better than waitForLoadState('networkidle').
3. Start waiting before the click. Assert status and JSON.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [resp] = await Promise.all([
  page.waitForResponse((r) => r.url().includes('/apply') && r.status() === 201),
  page.getByRole('button', { name: 'Submit' }).click(),
]);
expect((await resp.json()).id).toBeTruthy();
```

5. Matching the wrong URL (analytics) makes the test pass while the save failed.

34. What is APIRequestContext?

1. Definition you should say first: HTTP requests without a browser, for setup or API tests. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. APIRequestContext is HTTP without a browser. Use it to seed, login, and assert APIs.
3. Create the applicant via API, then open UI to verify the list. Faster and less flake.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
test('list shows seeded job', async ({ request, page }) => {
  await request.post('/api/jobs', { data: { title: 'Playwright QA', ctc: 30 } });
  await page.goto('/jobs');
  await expect(page.getByRole('cell', { name: 'Playwright QA' })).toBeVisible();
});
```

5. Do not put production bearer tokens in repo. Use env and a test tenant.

35. How do you test downloads?

1. Definition you should say first: Wait for download event and save the file. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Wait for the download event, then save to a unique path. Assert suggestedFilename and file bytes if needed.
3. Enable acceptDownloads (default in Playwright Test). Clean the folder in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export PDF' }).click(),
]);
expect(download.suggestedFilename()).toMatch(/interview-tosca\.pdf/);
await download.saveAs('test-results/tosca.pdf');
```

5. A shared Downloads directory across workers overwrites files.

36. How do you test uploads?

1. Definition you should say first: setInputFiles on the file input. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. setInputFiles works on <input type=file>, including hidden inputs. Chrome extensions cannot do this; Playwright tests can.
3. Pass a path Playwright can read on the agent. For multiple files, pass an array.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Resume').setInputFiles('fixtures/RaviKumar.pdf');
await page.locator('input[type=file]').setInputFiles(['a.pdf', 'b.pdf']);
```

5. Custom dropzones that ignore the input need DataTransfer dispatch. Prefer asking devs for an input.

37. Can Playwright set a file input?

1. Definition you should say first: Yes in tests; a Chrome extension cannot. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. setInputFiles works on <input type=file>, including hidden inputs. Chrome extensions cannot do this; Playwright tests can.
3. Pass a path Playwright can read on the agent. For multiple files, pass an array.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Resume').setInputFiles('fixtures/RaviKumar.pdf');
await page.locator('input[type=file]').setInputFiles(['a.pdf', 'b.pdf']);
```

5. Custom dropzones that ignore the input need DataTransfer dispatch. Prefer asking devs for an input.

38. How do you handle a new tab?

1. Definition you should say first: Wait for a popup page event. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Clicks that open a tab fire a popup event. Wait for it before talking to the new page.
3. Close extra pages in teardown so workers do not leak.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [tab] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Privacy' }).click(),
]);
await expect(tab).toHaveTitle(/Privacy/);
```

5. If the site uses target=_blank plus noopener, you still get a Page object via the event.

39. How do you handle a dialog?

1. Definition you should say first: page.on('dialog') accept or dismiss. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Native alert/confirm/prompt are dialog events. Register the listener before the action.
3. Accept or dismiss explicitly. Unhandled dialogs stall the test.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
page.once('dialog', async (d) => {
  expect(d.message()).toContain('Delete');
  await d.accept();
});
```

```
await page.getByRole('button', { name: 'Delete' }).click();
```

5. A leftover listener from a previous test can auto-accept a later dialog.

40. How do you run headed on a laptop?

1. Definition you should say first: headed: true or --headed. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Headed is for local debug. CI should be headless, with retries 1-2, HTML report, and traces on failure.
3. Use xvfb only if a dependency needs a display. GitHub Actions ubuntu works headless.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test --headed
npx playwright test --project=chromium --reporter=html
# CI: retries: 2, workers: 2, forbid only
```

5. Do not debug only headed; some timing bugs appear only headless.

41. How do you run in CI?

1. Definition you should say first: headless, HTML report, retries 1-2. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Headed is for local debug. CI should be headless, with retries 1-2, HTML report, and traces on failure.
3. Use xvfb only if a dependency needs a display. GitHub Actions ubuntu works headless.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test --headed
npx playwright test --project=chromium --reporter=html
# CI: retries: 2, workers: 2, forbid only
```

5. Do not debug only headed; some timing bugs appear only headless.

42. What is a flake retry?

1. Definition you should say first: Re-run a failed test; fix the flake, do not hide it forever. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.
3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

43. What is a screenshot on failure?

1. Definition you should say first: Configured in use: screenshot: 'only-on-failure'. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.
3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

44. What is video on failure?

1. Definition you should say first: use.video: 'retain-on-failure'. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.
3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

45. What is a soft assertion?

1. Definition you should say first: expect.soft that does not stop the test immediately. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. expect.soft continues after failure so you collect many UI bugs in one run.
3. Use for long forms. End with a hard expect or the test may exit 0 incorrectly if you forget.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect.soft(page.getByLabel('City')).toHaveValue('Bengaluru');
await expect.soft(page.getByLabel('Pin')).toHaveValue('560001');
await expect(page.getByRole('button', { name: 'Save' })).toBeEnabled();
```

5. All-soft tests that never hard-fail hide broken setup.

46. What is test.describe?

1. Definition you should say first: A group of tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

47. What is test.beforeEach?

1. Definition you should say first: Runs before every test in the group. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

48. What is a tag in Playwright?

1. Definition you should say first: test.info().annotations or grep @smoke. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

49. How do you run only smoke?

1. Definition you should say first: npx playwright test --grep @smoke Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
```

```
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

50. What is a locator filter?

1. Definition you should say first: Narrow a locator by text or another locator. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.

3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```

5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

51. What is getByLabel?

1. Definition you should say first: Find a control via its label text. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByLabel uses the associated label. getByPlaceholder is weaker because placeholders disappear when filled and are not always accessible.

3. Prefer label. If the control has no label, that is an a11y bug worth raising.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Current CTC (LPA)').fill('28');
await page.getByPlaceholder('dd/mm/yyyy').fill('03/09/2026');
```

5. Placeholder-only locators break when UX writes 'e.g. 30' instead of the old text.

52. What is getByPlaceholder?

1. Definition you should say first: Find an input by placeholder. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByLabel uses the associated label. getByPlaceholder is weaker because placeholders disappear when filled and are not always accessible.

3. Prefer label. If the control has no label, that is an a11y bug worth raising.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Current CTC (LPA)').fill('28');
await page.getByPlaceholder('dd/mm/yyyy').fill('03/09/2026');
```

5. Placeholder-only locators break when UX writes 'e.g. 30' instead of the old text.

53. What is a frame locator?

1. Definition you should say first: Enter an iframe without driver.switchTo. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. frameLocator enters an iframe without driver.switchTo. Locators pierce open shadow roots by default.

3. Chain frameLocator for nested frames (SSO widgets). Closed shadow roots need a test id on the host.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const stripe = page.frameLocator('iframe[name=stripe]');
await stripe.getByPlaceholder('Card number').fill('4242424242424242');
await page.locator('custom-widget').getByRole('button', { name: 'Pay' }).click();
```

5. Wrong frame = strict timeout. Confirm the iframe selector in the trace snapshot.

54. How do you work with Shadow DOM?

1. Definition you should say first: Locators pierce open shadow roots. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. frameLocator enters an iframe without driver.switchTo. Locators pierce open shadow roots by default.

3. Chain frameLocator for nested frames (SSO widgets). Closed shadow roots need a test id on the host.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const stripe = page.frameLocator('iframe[name=stripe]');
await stripe.getByPlaceholder('Card number').fill('4242424242424242');
await page.locator('custom-widget').getByRole('button', { name: 'Pay' }).click();
```

5. Wrong frame = strict timeout. Confirm the iframe selector in the trace snapshot.

55. What is fill vs type?

1. Definition you should say first: fill sets the value; type sends key events. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. fill sets the value and fires input events. type/pressSequentially sends keys. Use the latter when the app listens to each key (OTP, masks).

3. Clear then fill for React controlled inputs if fill alone does not stick.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill('9876543210');
await page.getByLabel('OTP').pressSequentially('123456', { delay: 50 });
```

5. type into a filled field appends and fails validation. fill replaces.

56. When do you use pressSequentially?

1. Definition you should say first: When the app listens to each key. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. fill sets the value and fires input events. type/pressSequentially sends keys. Use the latter when the app listens to each key (OTP, masks).

3. Clear then fill for React controlled inputs if fill alone does not stick.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill('9876543210');
await page.getByLabel('OTP').pressSequentially('123456', { delay: 50 });
```

5. type into a filled field appends and fails validation. fill replaces.

57. What is click force?

1. Definition you should say first: Bypasses some checks; use rarely. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. force: true clicks even if hidden. hover opens menus that need pointer over.

3. Prefer waiting for the overlay to hide. Use hover then click the menuitem role.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Profile' }).hover();
await page.getByRole('menuitem', { name: 'Logout' }).click();
// last resort: await locator.click({ force: true });
```

5. Force-clicking a covered button submits nothing. The trace will show the intercepting node.

58. What is hover?

1. Definition you should say first: Move the mouse for menus. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. force: true clicks even if hidden. hover opens menus that need pointer over.

3. Prefer waiting for the overlay to hide. Use hover then click the menuitem role.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Profile' }).hover();
await page.getByRole('menuitem', { name: 'Logout' }).click();
// last resort: await locator.click({ force: true });
```

5. Force-clicking a covered button submits nothing. The trace will show the intercepting node.

59. How do you select a dropdown?

1. Definition you should say first: selectOption on a select, or click options in a custom combo. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.

3. Always assert after mutation. Counting before the table fetch completes is a race.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

60. How do you check a checkbox?

1. Definition you should say first: locator.check(). Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.
3. Always assert after mutation. Counting before the table fetch completes is a race.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

61. How do you read text?

1. Definition you should say first: locator.innerText() or toHaveText. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.
3. Always assert after mutation. Counting before the table fetch completes is a race.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

62. How do you count rows?

1. Definition you should say first: locator.count() or toHaveCount. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.
3. Always assert after mutation. Counting before the table fetch completes is a race.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

63. What is a clock in Playwright?

1. Definition you should say first: Fake timers for time-dependent UI. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. The clock API fakes timers, dates, and animations so expiry banners and OTPs are deterministic.
3. Install clock before the page uses Date.now. Run only the timers you need.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.clock.install({ time: new Date('2026-09-03T10:00:00') });
await page.goto('/offer');
await page.clock.fastForward('01:00:00');
await expect(page.getByText('Offer expired')).toBeVisible();
```

5. Faking time while a real setTimeout from analytics is running can hang. Pause and inspect.

64. What is authentication setup project?

1. Definition you should say first: A project that logs in once and writes storageState. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.
3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

65. How do you test localization?

1. Definition you should say first: A project per locale. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

66. How do you test a mobile viewport?

1. Definition you should say first: Device descriptor in a project. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

67. Does Playwright drive a real phone?

1. Definition you should say first: It emulates viewport and UA; real devices need another stack. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

68. What is Playwright for Python vs Node?

1. Definition you should say first: Same browser engine; this kit uses Python in places and Node is common in web teams. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Playwright is Microsoft's open-source library for reliable end-to-end tests. One API covers Chromium, Firefox, and WebKit, with browsers installed via `npx playwright install`.
3. In interviews, contrast it with Selenium: auto-wait, traces, API mocking, and test fixtures are built in. Node/TS is the default at most web teams; Python uses the same browser engine.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npm init playwright@latest
npx playwright install
npx playwright test --project=chromium
npx playwright show-report
```

5. If you only say 'it automates browsers' you fail the next question. Name locators, contexts, traces, and storageState.

69. How do you install browsers?

1. Definition you should say first: playwright install. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.
3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

70. What is a browserType.launch?

1. Definition you should say first: Lower-level API vs Playwright Test fixtures. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.
3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

71. When do you use persistent context?

1. Definition you should say first: To keep a profile, as JobKit does for LinkedIn session. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.
3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

72. What is CDP?

1. Definition you should say first: Chrome DevTools Protocol; Playwright can connect over it. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.
3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

73. Why not click Submit in JobKit?

1. Definition you should say first: Sites forbid bots; you submit yourself after fill. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. JobKit Fill never clicks Submit. Sites forbid unattended apply bots. You review and submit.
3. Tests that exercise a public apply form should still not fire production applications. Use a dummy tenant.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill(phone);
await expect(page.getByRole('button', { name: 'Submit' })).toBeEnabled();
// do not: await page.getByRole('button', { name: 'Submit' }).click();
```

5. If a recruiter asks you to 'fully automate Easy Apply', decline. It violates LinkedIn terms and this kit's rules.

74. What is a POM?

1. Definition you should say first: Page Object Model: locators and actions in a class. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.
3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
```

```
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

75. Playwright and POM?

1. Definition you should say first: Use fixtures plus small page objects; avoid huge base classes. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.
3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

76. What is a component locator?

1. Definition you should say first: A locator root for a widget, then getByRole inside it. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.
3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

77. How do you avoid xpath soup?

1. Definition you should say first: Prefer role, label, and test id. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.
3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

78. What is an accessible name?

1. Definition you should say first: The name screen readers use; getByRole uses it. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.
3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
```

```
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

79. How do you test file download names?

1. Definition you should say first: download.suggestedFilename(). Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Wait for the download event, then save to a unique path. Assert suggestedFilename and file bytes if needed.

3. Enable acceptDownloads (default in Playwright Test). Clean the folder in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export PDF' }).click(),
]);
expect(download.suggestedFilename()).toMatch(/interview-tosca\.pdf/);
await download.saveAs('test-results/tosca.pdf');
```

5. A shared Downloads directory across workers overwrites files.

80. How do you wait for URL?

1. Definition you should say first: expect(page).toHaveURL. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

81. How do you wait for title?

1. Definition you should say first: expect(page).toHaveTitle. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

82. What is networkidle?

1. Definition you should say first: Wait until no connections; often too brittle, prefer response waits. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

83. What is domcontentloaded?

1. Definition you should say first: HTML parsed; assets may still load. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

84. What is load?

1. Definition you should say first: Window load event. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

85. Which wait is default for goto?

1. Definition you should say first: Usually load; you can pass waitUntil. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

86. How do you debug a test?

1. Definition you should say first: PWDEBUG=1 or --debug and Trace Viewer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PWDEBUG=1 or --debug opens Inspector. --ui is watch mode with time travel. Traces work after CI failures.

3. Reproduce with --repeat-each=5 on a flake. Step through locators in Inspector.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test tests/apply.spec.ts:12 --debug
npx playwright test --ui
$env:PWDEBUG=1; npx playwright test
```

5. Debugging only on your laptop misses CI viewport and timezone issues. Match CI config locally.

87. What is UI mode?

1. Definition you should say first: playwright test --ui for watch and time travel. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PWDEBUG=1 or --debug opens Inspector. --ui is watch mode with time travel. Traces work after CI failures.

3. Reproduce with --repeat-each=5 on a flake. Step through locators in Inspector.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test tests/apply.spec.ts:12 --debug
npx playwright test --ui
$env:PWDEBUG=1; npx playwright test
```

5. Debugging only on your laptop misses CI viewport and timezone issues. Match CI config locally.

88. How do you seed test data?

1. Definition you should say first: APIRequestContext in beforeEach or a factory. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. APIRequestContext is HTTP without a browser. Use it to seed, login, and assert APIs.
3. Create the applicant via API, then open UI to verify the list. Faster and less flake.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
test('list shows seeded job', async ({ request, page }) => {
  await request.post('/api/jobs', { data: { title: 'Playwright QA', ctc: 30 } });
  await page.goto('/jobs');
  await expect(page.getByRole('cell', { name: 'Playwright QA' })).toBeVisible();
});
```

5. Do not put production bearer tokens in repo. Use env and a test tenant.

89. How do you hide secrets in CI?

1. Definition you should say first: Env vars, not committed .env. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Secrets live in CI variables, not in specs or committed .env.
3. Read process.env.QA_PASSWORD. Fail fast if missing. Mask in logs.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const password = process.env.QA_PASSWORD;
if (!password) throw new Error('QA_PASSWORD missing');
await page.getByLabel('Password').fill(password);
```

5. Traces can capture typed passwords. Use the password field and avoid extra screenshots of the vault.

90. What is a visual snapshot?

1. Definition you should say first: Screenshot compare with toHaveScreenshot. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.
3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

91. Why do visual tests flake?

1. Definition you should say first: Fonts, animation, and time; mask dynamic regions. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.
3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

92. What is a mask in screenshots?

1. Definition you should say first: Hide volatile locators before compare. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.
3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

93. How do you test accessibility?

1. Definition you should say first: axe via a Playwright plugin or extra library. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.

94. What is a console error test?

1. Definition you should say first: Listen to pageon console and fail on error if the team agrees. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.

95. How do you handle basic auth?

1. Definition you should say first: httpCredentials in context. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.

96. How do you set geolocation?

1. Definition you should say first: context.grantPermissions and setGeolocation. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.

97. How do you block ads?

1. Definition you should say first: route abort on ad hosts. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
```

```
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.

98. What is a har file?

1. Definition you should say first: Recorded network log you can replay. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.

3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.